Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Tasks to be Performed

1. Download and understand the dataset.
2. Perform Exploratory Data Analysis (EDA).
3. Preprocess the dataset.
4. Implement a Single Layer Perceptron from scratch.
5. Train the model using the perceptron learning rule.
6. Evaluate the classifier using standard performance metrics.
7. Analyze the effect of different learning rates.
8. Compare the implementation with Scikit-learn's Perceptron (optional).

3. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T \mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

4. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

5. Dataset Description

Dataset: Banknote Authentication Dataset
Source: UCI Machine Learning Repository
Download Link:
`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

6. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Expected Output
Dataset summary and statistical information.

Task 2: Exploratory Data Analysis
Generate

- Histograms

- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.
- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

7. Mandatory Plots

For each plot, insert the corresponding figure and provide a brief interpretation (2–3 sentences).

7.1 Feature Histograms

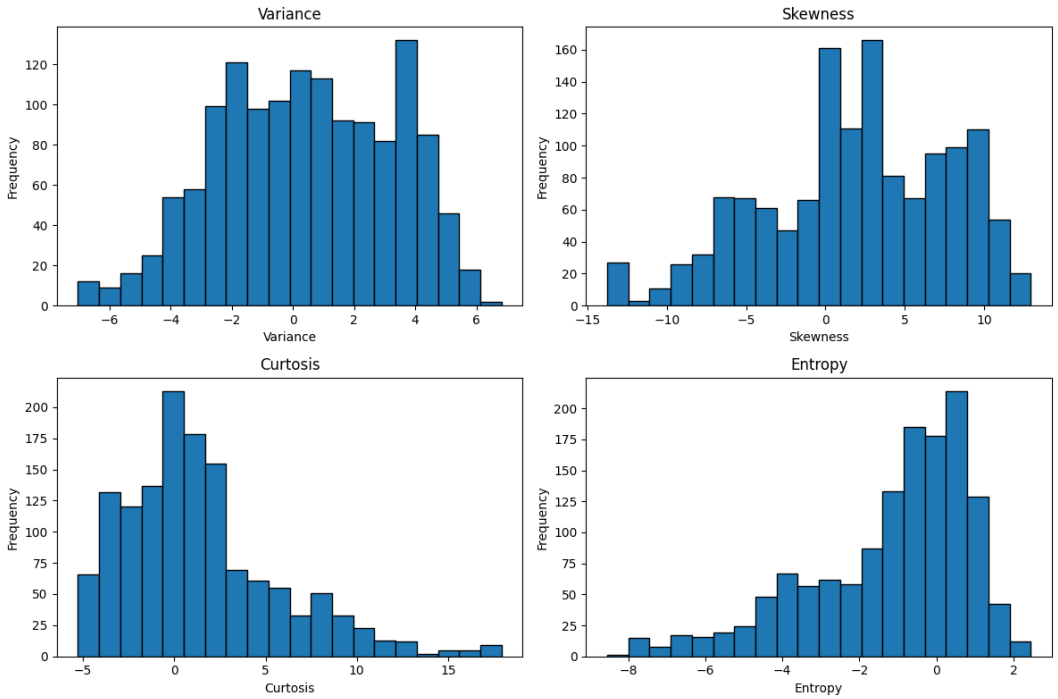


Figure 1: Feature Histograms

Interpretation variance has good spread and is fairly well distributed. Skewness has long spread from -14 to 12. Curtosis has most of its value at the right so it is right skewed. Entropy has most of its value at the begining so it is left skewed

7.2 Correlation Heatmap

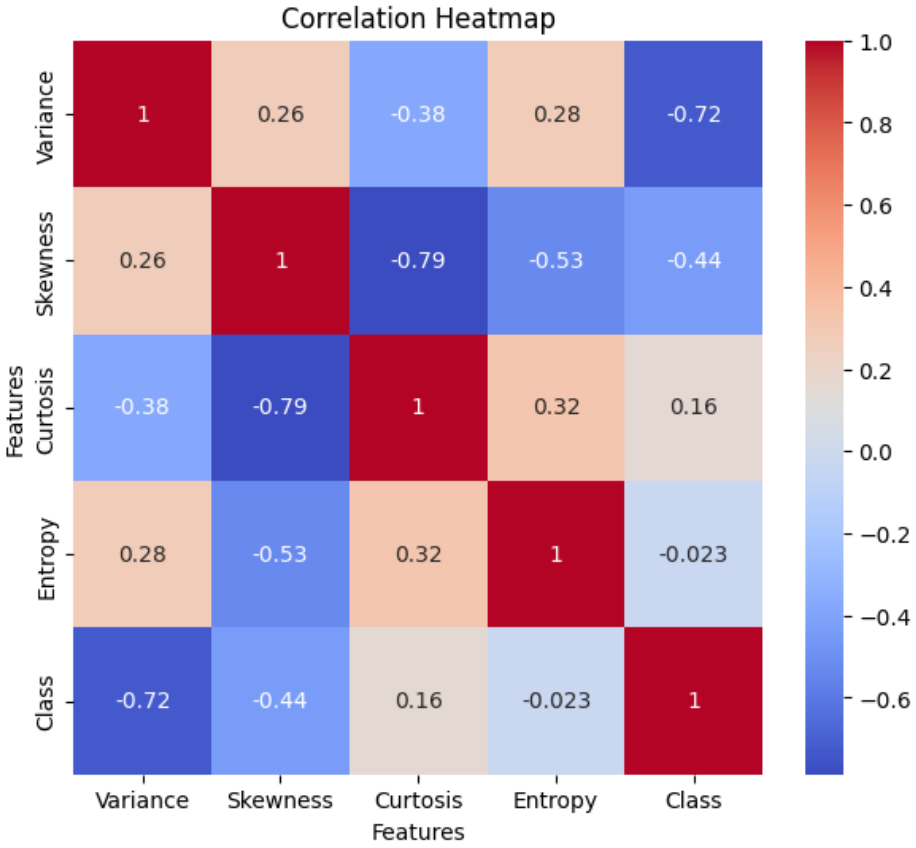


Figure 2: Correlation Heatmap

Interpretation variance has strong negative relationship with class and has weak relaationship with all other features. Skeweness has moderate negative relationship with class and entropy, it has strong relationship with curtosis. Curtosis has weak positive rlationship with class. Entropy has very weak negative relationship with the class

7.3 Scatter Plot

Figure 3: Scatter Plot

Interpretation There is only small overlap between the classes so the two classes can seperated using variance and skewness

7.4 Training Error vs Epoch

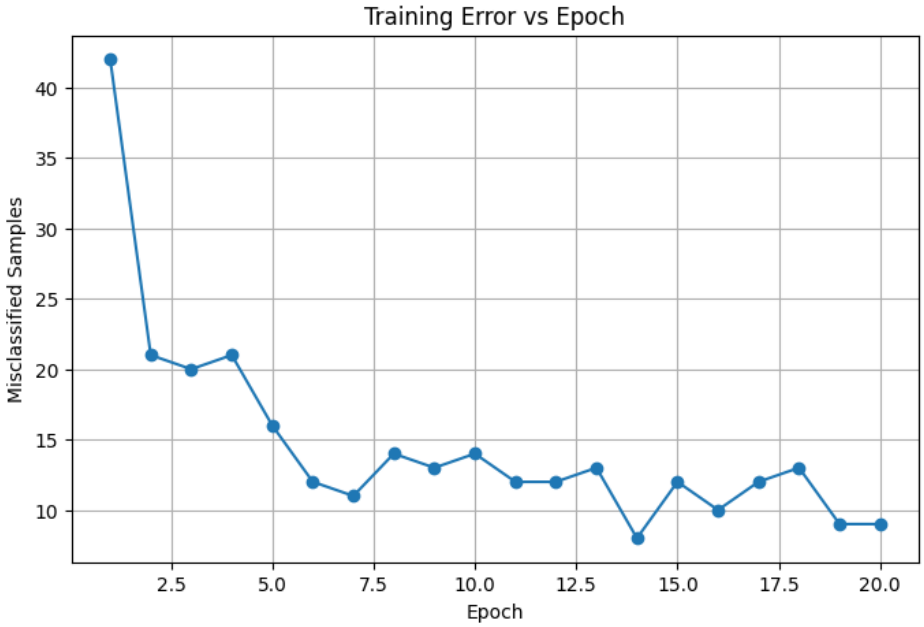


Figure 4: Training Error vs Epoch

Interpretation the training error is reducing as the number of epochs are increasing

7.5 Weight Evolution

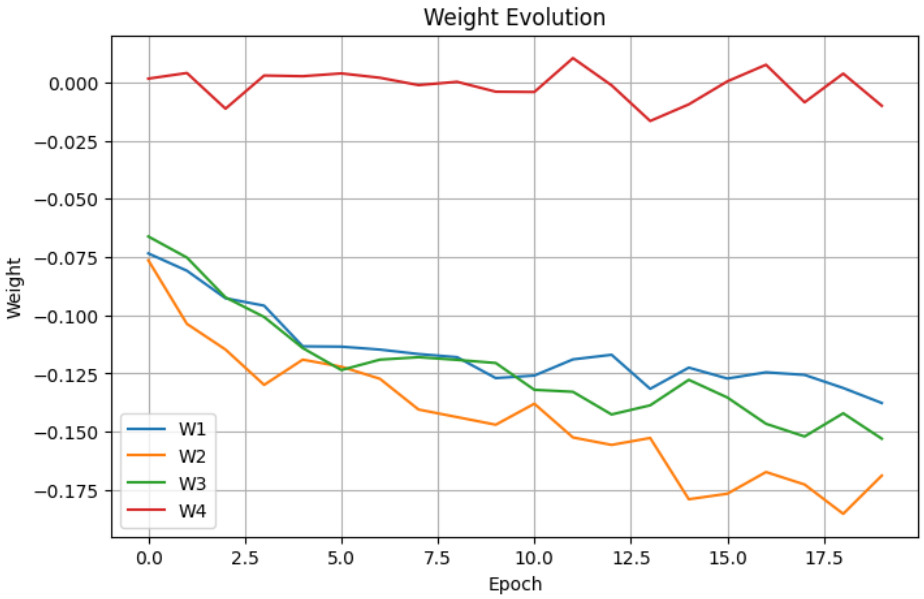


Figure 5: Weight Evolution

Interpretation W2 has the highest magnitude indicating it contributes highest to the decision boundary. while w4 remains close to zero after every iteration indicating it contributes very less

7.6 Bias Evolution

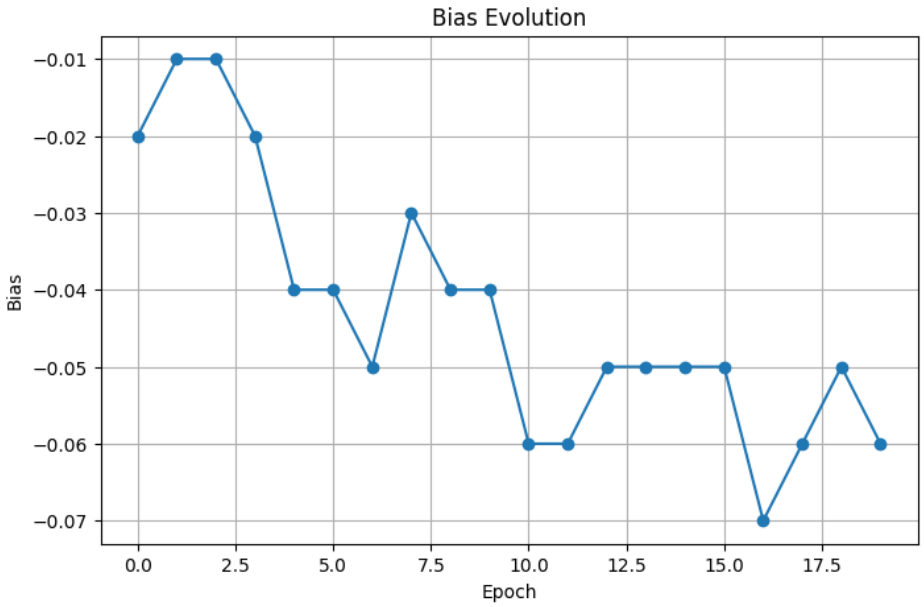


Figure 6: Bias Evolution

Interpretation Intially there are fluctuations in the bias and after some epochs there are very small fluctuations

7.7 Confusion Matrix

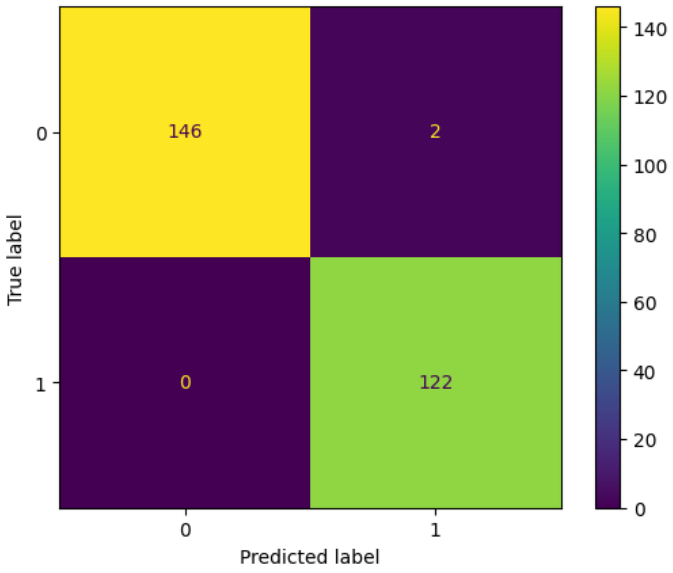


Figure 7: Confusion Matrix

Interpretation only two samples are incorrectly identified so the model is performing very good

7.8 Learning Rate Comparison

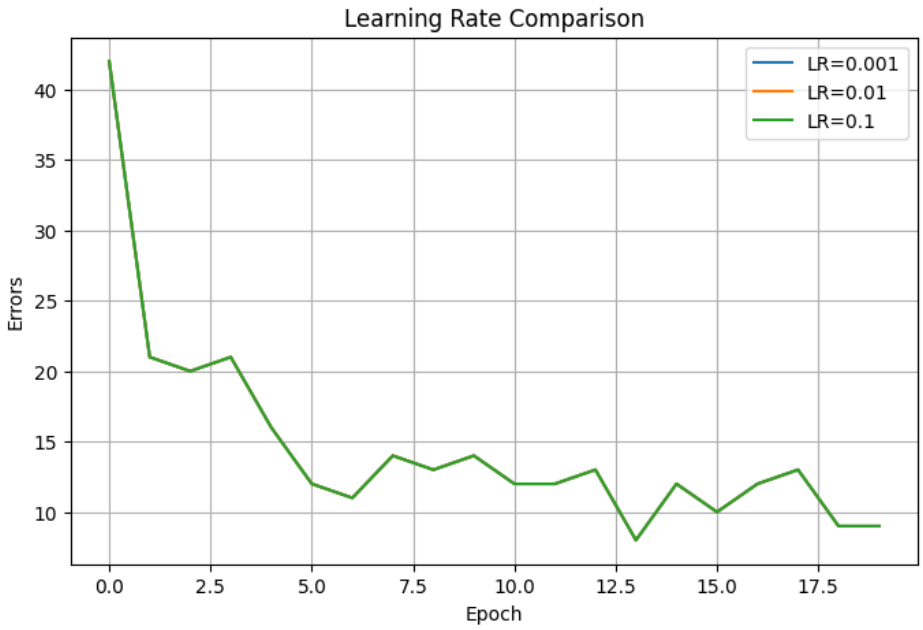


Figure 8: Learning Rate Comparison

Interpretation All the three learning rates are overlappping which means the model performes same with all the learning rates

8. Performance Tables

Training Summary

Dataset Size	1372 Samples
Train/Test Split	80% Training, 20% Testing
Learning Rate	0.01
Epochs	20
Final Weights	[-0.1376, -0.1688, -0.1530, -0.0100]
Final Bias	-0.06
Accuracy	99.26%
Precision	98.39%
Recall	100.00%
F1-score	99.19%

Epoch-wise Learning

Epoch	Errors	Weight 1	Weight 2	Bias
1	42	-0.0734	-0.0764	-0.02
2	21	-0.0808	-0.1037	-0.01
3	20	-0.0927	-0.1147	-0.01
4	21	-0.0958	-0.1299	-0.02
5	16	-0.1133	-0.1190	-0.04
6	12	-0.1134	-0.1221	-0.04
7	11	-0.1147	-0.1273	-0.05
8	14	-0.1166	-0.1404	-0.03
9	13	-0.1180	-0.1437	-0.04
10	14	-0.1269	-0.1470	-0.04
11	12	-0.1259	-0.1380	-0.06
12	12	-0.1189	-0.1524	-0.06
13	13	-0.1169	-0.1556	-0.05
14	8	-0.1316	-0.1527	-0.05
15	12	-0.1225	-0.1790	-0.05
16	10	-0.1271	-0.1766	-0.05
17	12	-0.1245	-0.1673	-0.07
18	13	-0.1256	-0.1726	-0.06
19	9	-0.1312	-0.1852	-0.05
20	9	-0.1376	-0.1688	-0.06

9. Discussion

This section presents the observations and answers to the analytical questions based on the experiment.

Q1. Compare the Step and Sigmoid activation functions. Why is the Step Activation Function not used in modern Deep Learning?

Step function produces only binary output and has sharp threshold at z=0 and is not differentiable at z=0. While Activation function producces continous output from 0 to 1 and sigmoid function has a smooth ad differentiable function

Step function is not used in modern deep learning as is not differentiable at z=0

Q2. Compare the implementation with Scikit-learn's Perceptron.

In Scikit-learn the perceptron is pre built and also has pre- built functions.

Q3. Compare the performance for learning rates 0.001, 0.01 and 0.1.

In the learning rate comparison graph all are overlapping so all of them have same performance

Q4. Why can't a Single Layer Perceptron solve the XOR problem?

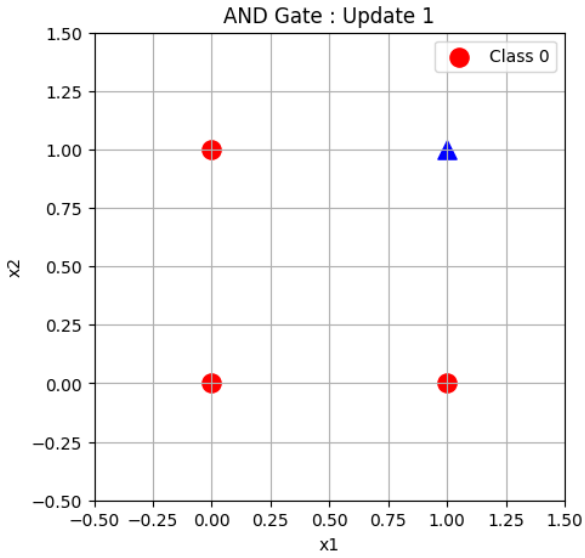
XOR is a non linear function and single perceptron can be used only for linear problem

Q5. Explain the effect of feature normalization on model convergence.

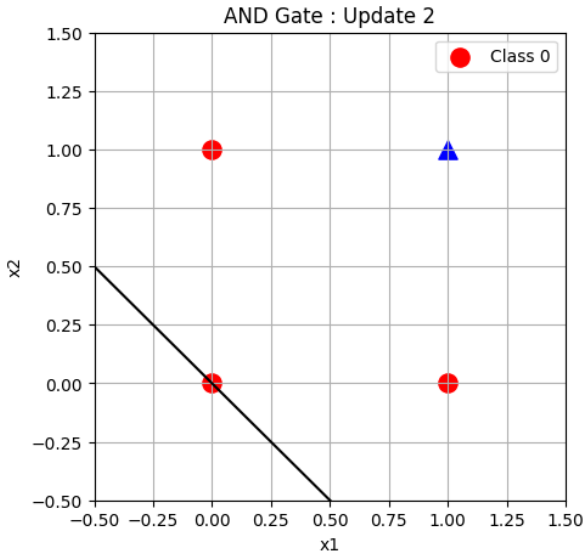
Feature normalization scales all input features to a common range. which reduces the chance of model considering large values as more important and small values as less important

10.Decision Boundary After Each Weight Update

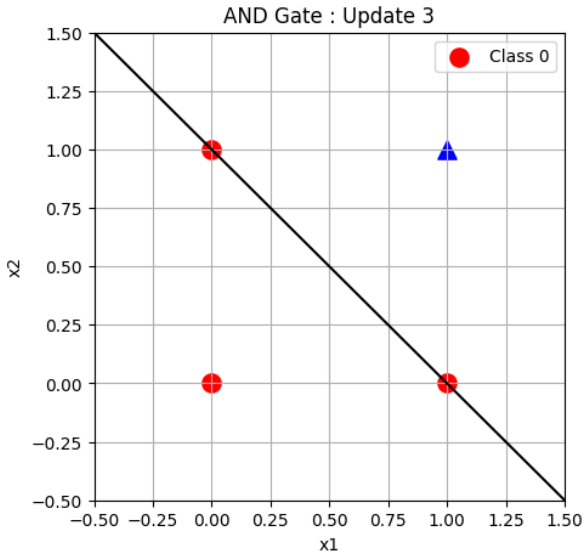
A. AND Gate (11 Weight Updates)



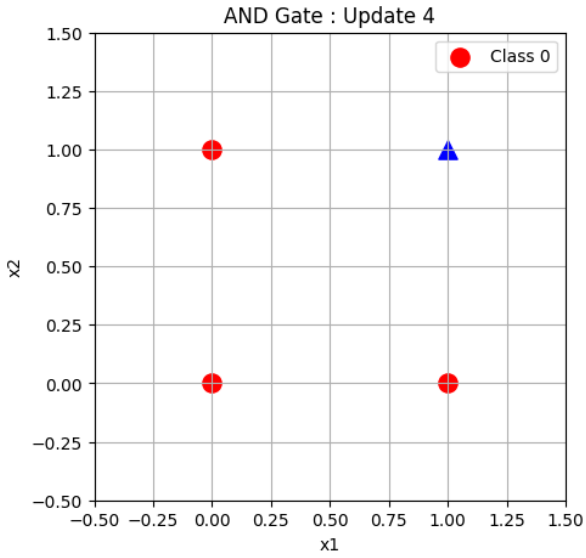
(a) Update 1



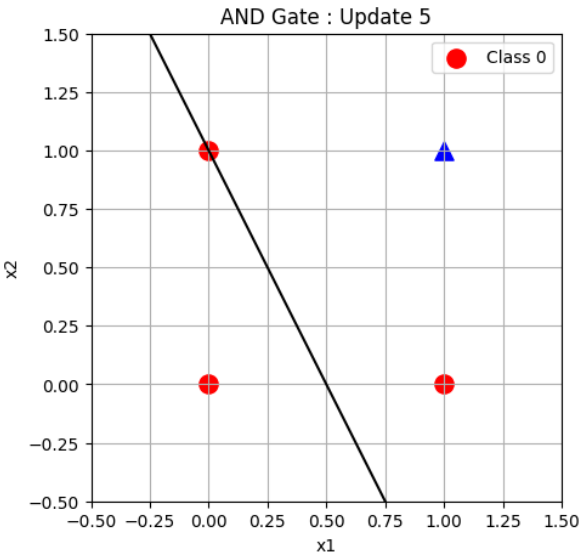
(b) Update 2



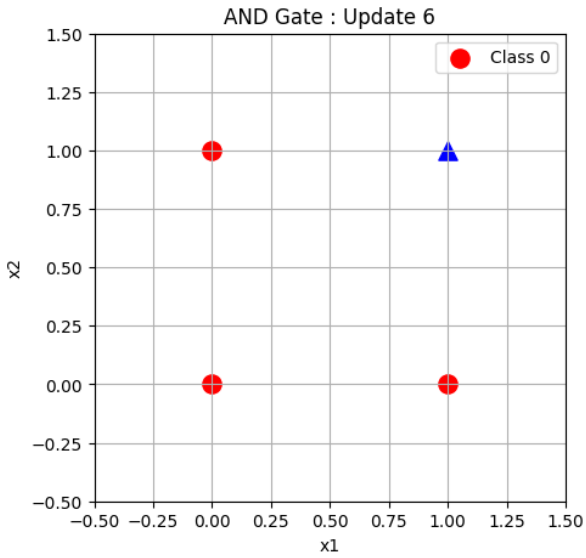
(a) Update 3



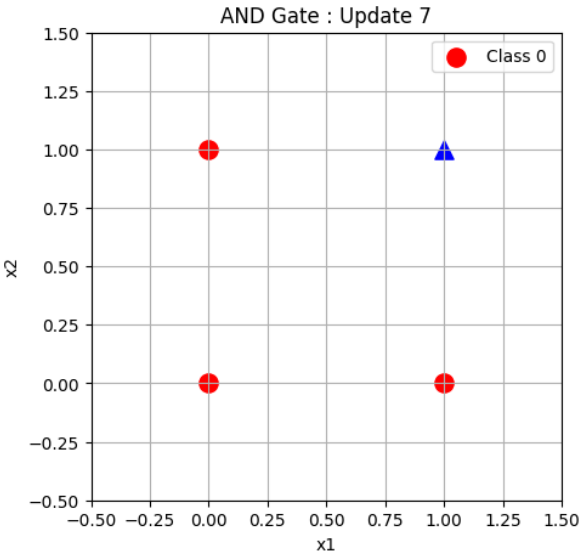
(b) Update 4



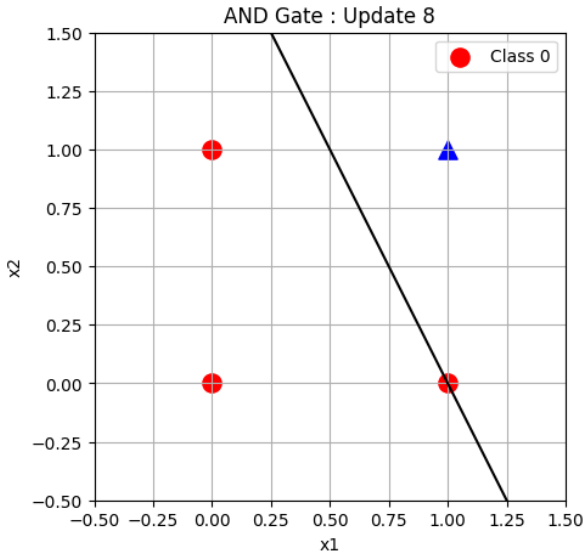
(a) Update 5



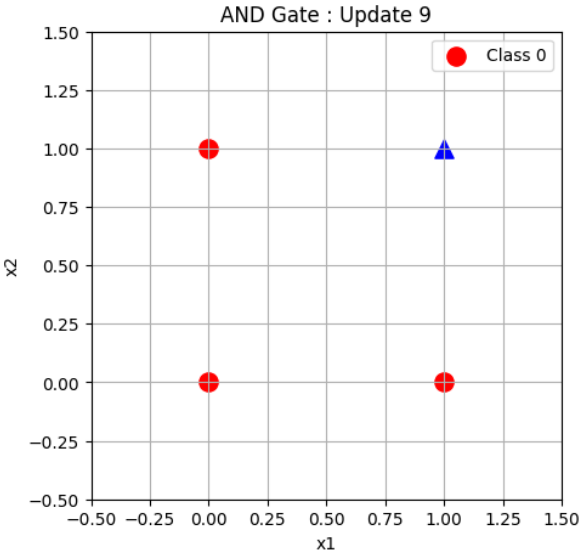
(b) Update 6



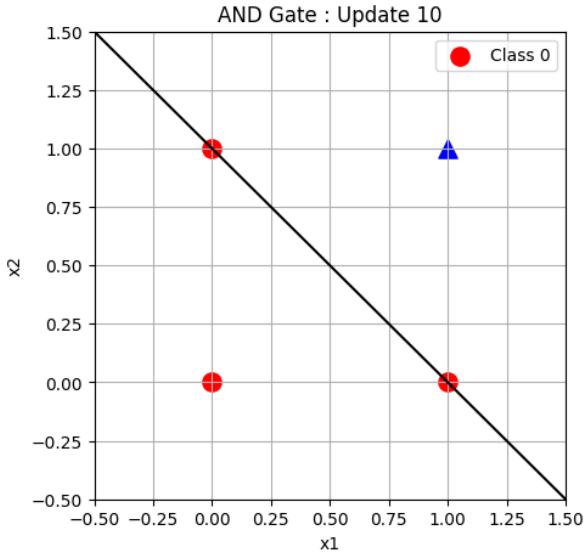
(a) Update 7



(b) Update 8



(a) Update 9



(b) Update 10

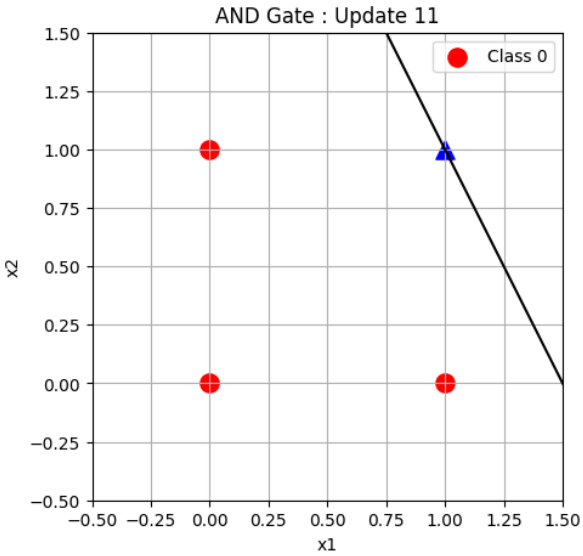
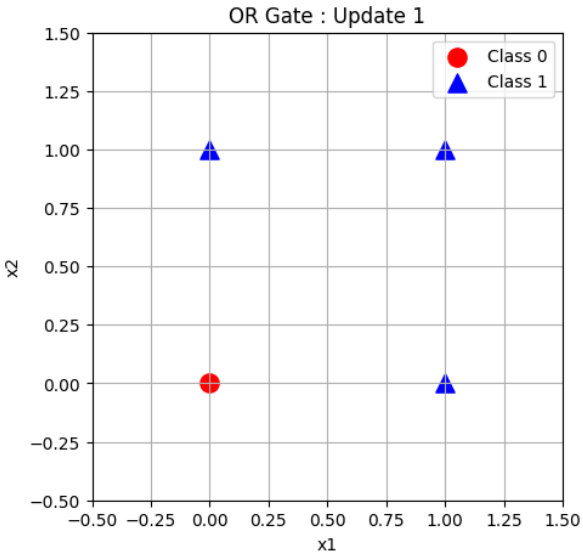
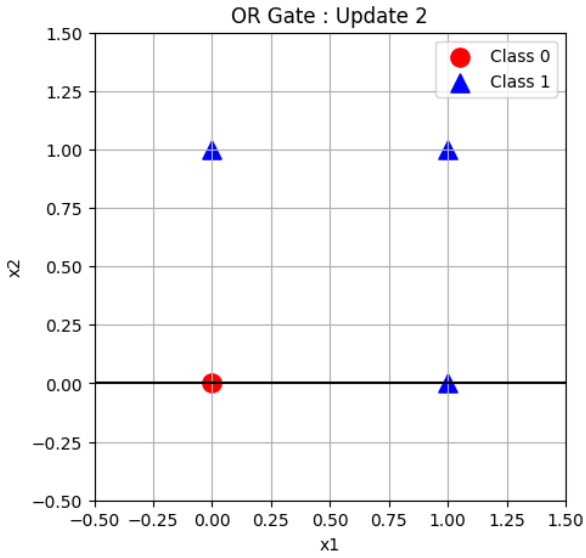


Figure 14: Update 11

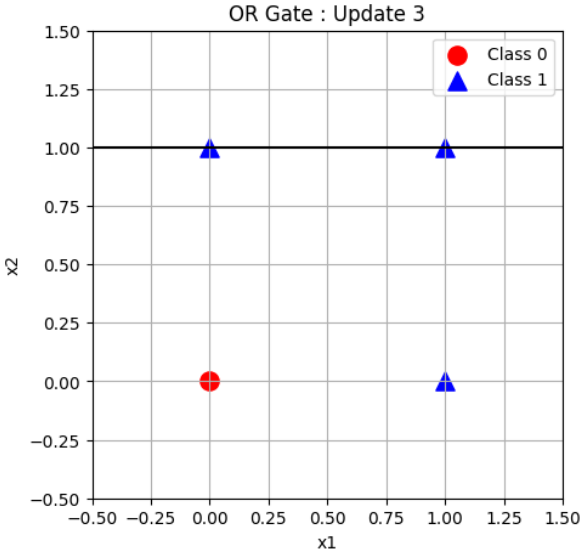
B. OR Gate (5 Weight Updates)



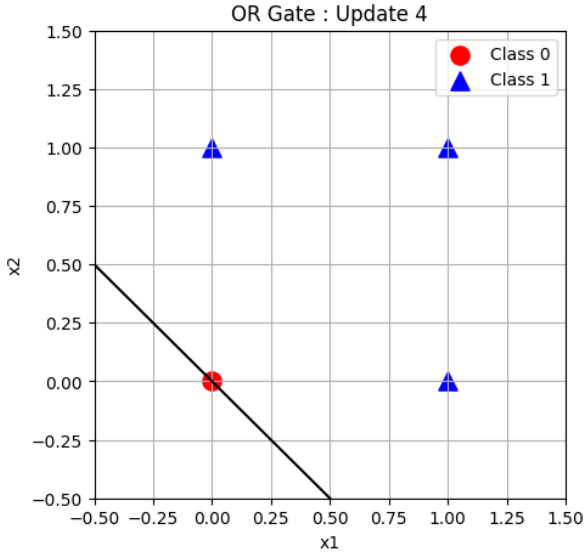
(a) Update 1



(b) Update 2



(a) Update 3



(b) Update 4

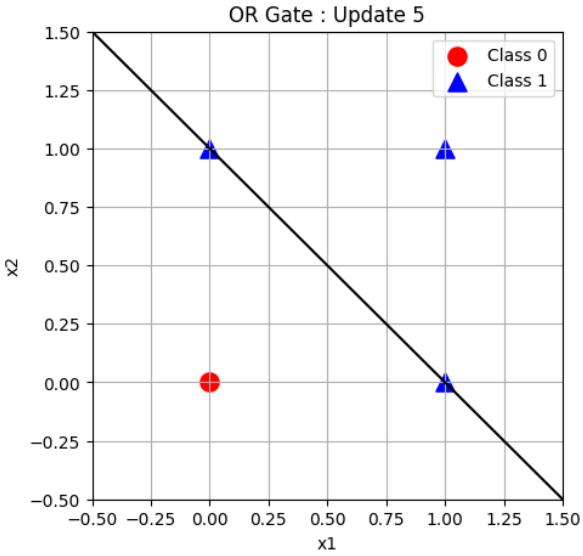
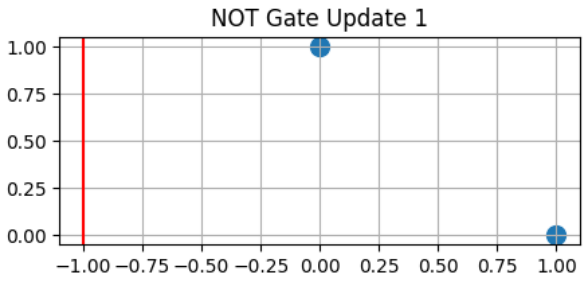
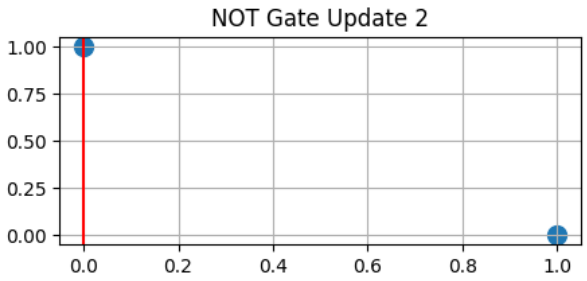


Figure 17: Update 5

C. NOT Gate (2 Weight Updates)



(a) Update 1



(b) Update 2

12. Source Code Repository

The complete implementation of this experiment, including data preprocessing, model construction, training, evaluation, and hyperparameter optimization, is available in the following GitHub repository:

GitHub Repository:
`https://github.com/shaliniparvathareddy/Deep_learning_perceptron`